

AI Agent 开发圣经

从 LLM 到 Agent (2026)

完整版教材工程版 | LLM · Prompt · RAG · Agent · Tools · LangGraph · MCP

Generated from the latest book/ source.

目录

- 01. 前言：为什么需要 AI Agent 开发圣经
- 02. AI 到底是什么：从概率预测到工程边界
- 03. Transformer 是什么：Attention、QKV 与生成机制
- 04. Prompt工程（如何控制AI）
- 05. RAG（检索增强生成）
- 06. Agent（AI如何真正干活）
- 07. Tools & Function Calling（让AI真正动手）
- 08. LangGraph（AI Agent编排系统）
- 09. MCP（Model Context Protocol）
- 10. AI Agent实战项目（电商 / CRM / 教师）
- 11. AI面试终极宝典（LLM / RAG / Agent / MCP）

Chapter 01

前言：为什么需要 AI Agent 开发圣经

Story · Technical · Diagram · Code · Engineering · Interview

前言：为什么需要 AI Agent 开发圣经

Story Explanation

想象一家公司的运营同学每天要打开十几个系统：查销售数据、读客户反馈、生成日报、同步团队进展。传统脚本能处理固定流程，但一旦问题变成“帮我判断今天异常在哪里，并给出下一步动作”，脚本就开始吃力。

AI Agent

的价值，就是把语言理解、工具调用、状态管理和反馈循环组合成一个能协助完成复杂任务的工程系统。

Technical Explanation

AI Agent 不是一个单独模型，而是围绕目标构建的系统：LLM 负责理解和生成，工具负责访问外部世界，记忆负责保存状态，编排层负责控制步骤，评估层负责判断质量。一本 Agent 教材必须同时讲清模型能力和工程边界，否则读者只能做演示，难以做生产系统。

Mermaid Diagram

```
graph LR
    N0["目标"] --> N1["LLM"]
    N1 --> N2["工具"]
    N2 --> N3["状态"]
    N3 --> N4["评估"]
    N4 --> N5["人类确认"]
    N5 -- feedback --> N1
```

Python Code

```
from dataclasses import dataclass

@dataclass
class Milestone:
    name: str
    outcome: str

path = [
    Milestone("LLM Basics", "understand tokens and context"),
    Milestone("RAG", "ground answers in documents"),
    Milestone("Agent", "combine tools, memory, and control"),
]

for step, milestone in enumerate(path, 1):
    print(f'{step}. {milestone.name}: {milestone.outcome}')
```

See also: [example.py](#)

Engineering Use Case

为团队搭建一个“AI 工作流助理”，它能读取任务描述、选择模板、调用数据查询工具、生成阶段性报告，并在高风险步骤前请求人工确认。

Interview Questions

- 为什么说 Agent 是系统而不是模型？
- 一个 Agent 项目最早应该设计哪些边界？
- 如何判断一个 AI demo 是否能进入生产环境？

Quality Checklist

- 解释是否能被没有框架经验的开发者理解。
- 技术概念是否能落到输入、输出、状态、工具和评估。
- Mermaid 图是否表达了系统流向。
- Python 示例是否可独立运行。
- 工程案例是否说明真实业务价值。

Navigation

- [Previous](#)
- [Next](#)

Chapter 02

AI 到底是什么：从概率预测到工程边界

Story · Technical · Diagram · Code · Engineering · Interview

第1章：AI到底是什么？

本章目标（非常重要）

学完这一章，你必须能够回答：

- AI到底是不是“思考”？
- ChatGPT到底在做什么？
- AI和人类思维的区别
- 为什么AI会“胡说”
- 什么是“概率模型”

1. 一个真实世界的故事（理解AI本质）

想象你有一个超级实习生。

他有一个特点：

他看过全世界几乎所有的书、文章、代码和对话。

但他有一个致命限制：

他从来没有真正“理解”任何东西。

这个超级实习生很厉害。你问他产品方案，他能写；你问他 Python 代码，他能写；你让他总结一篇论文，他也能总结。

但你必须记住：他并不是像人一样理解世界，而是在过去看过的大量文本中寻找模式，然后生成一个“最像正确答案”的回答。

举个例子

你问他：

今天天气怎么样？

他的思考过程不是：

- 去查天气
- 看实时数据
- 理解天气系统

而是：

- 在他看过的所有文本中，找最相似的问题
- 找最常见的答案模式

- 拼接一个“最可能的回答”

如果你没有给他实时天气工具，他并不知道今天真正的天气。他只是根据语言模式推测一个看起来合理的回答。

这就是很多 AI 错误的根源：它生成的是“概率上像答案的文本”，不一定是“现实中真实发生的事实”。

2. 所以AI的本质是什么？

一句话：

AI = 基于历史数据的概率预测系统

更准确地说，现代大语言模型会根据当前输入、上下文和训练中学到的模式，预测下一个最可能出现的 token。

当它连续预测很多 token 时，我们就看到了完整的回答。

所以 ChatGPT 看起来像是在说话、解释、推理、写代码，但从底层看，它仍然是在做一件事：

根据上下文，不断预测下一个最可能的内容。

3. AI不是你以为的“智能”

很多人误解 AI：

- AI会思考
- AI有意识
- AI理解世界

这些都不对。

AI 不会像人一样拥有真实体验。它不知道“热”是什么感觉，也不知道“下雨”是什么触感。它能描述这些东西，是因为它在大量文本中见过人类如何描述这些东西。

AI真实能力是：

- 模式匹配
- 概率计算
- 语言生成

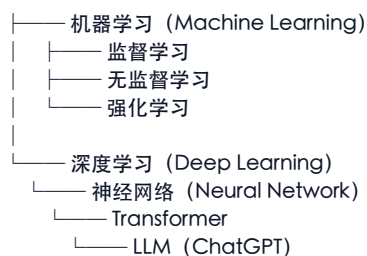
这三个能力非常强大，但它们不是人类意识。

你可以把 AI

理解成一个“超级模式预测器”。它能从海量数据中学习规律，并在新输入出现时生成最可能有用的输出。

4. AI的完整结构（必须记住）

AI（人工智能）



这张结构图非常重要。

AI 是一个大概念，机器学习是实现 AI 的一种重要方法。深度学习是机器学习中的一个分支，神经网络是深度学习的核心工具。Transformer 是现代大语言模型最关键的架构，而 LLM 是基于 Transformer 训练出来的大规模语言模型。

换句话说：

ChatGPT 不是“整个 AI”，它只是 AI 技术树上的一个重要分支。

5. AI 和传统程序有什么区别？

传统程序更像规则机器：

如果 A 发生，就执行 B

比如：

```
if temperature > 30:  
    print("天气很热")
```

只要条件一样，程序结果就一样。

AI 不一样。AI 更像概率判断：

看到输入 A，在上下文 C 下，生成最可能的结果 B

所以 AI 的输出可能会随着上下文、参数、模型版本和提示词变化。

这就是为什么 AI 工程不能只依赖“模型看起来很聪明”。真正的 AI 工程必须增加：

- 输入控制
- 上下文管理
- 工具调用
- 输出校验
- 事实验证
- 日志记录
- 人工确认

6. 为什么 AI 会“胡说”？

AI 胡说，通常不是因为它故意骗人。

更准确地说：

当模型缺少事实依据时，它仍然会尝试生成一个语言上合理的答案。

这就是幻觉 (hallucination) 。

比如你问：

某个不存在的论文作者是谁？

模型可能会生成一个看起来很像真实作者的名字。因为它学到的文本模式里，“论文标题 + 作者名字”是一种常见结构。

但这个答案可能完全不存在。

所以，AI 的幻觉来自两个原因：

- 它的目标是生成合理文本，而不是真正查询事实。
- 如果没有外部工具或资料，它无法确认答案是否真实。

7. 工程上应该怎么使用AI？

正确使用 AI 的方式不是问：

这个模型是不是足够聪明？

而是问：

我如何把模型放进一个可靠的系统里？

一个可靠的 AI 系统通常需要：

- 用 Prompt 明确任务和边界
- 用 RAG 提供可信资料
- 用工具查询实时数据
- 用结构化输出减少不稳定性
- 用评估集测试质量
- 用日志追踪每一步行为
- 在高风险动作前加入人工确认

这也是本书后续章节要解决的问题。

Mermaid Diagram

```
graph TD
    A["用户输入"] --> B["上下文"]
    B --> C["概率预测"]
    C --> D["生成回答"]
    D --> E["是否有事实依据?"]
    E -- "有" --> F["可信输出"]
    E -- "没有" --> G["可能幻觉"]
    G --> H["需要 RAG / 工具 / 人工确认"]
```

H → B

Python Code

下面这个例子不是在实现真正的 AI，而是用一个极简方式模拟“概率预测”的感觉。

```
from collections import Counter

history = [
    "AI is prediction",
    "AI is pattern matching",
    "AI is prediction",
    "LLM predicts the next token",
    "AI is probability",
]

words = []
for sentence in history:
    words.extend(sentence.lower().split())

counter = Counter(words)

print("Most common patterns:")
for word, count in counter.most_common(5):
    print(word, count)
```

See also: [example.py](#)

Engineering Use Case

假设你要做一个企业内部 AI 助手，用户问：

我们公司的报销政策是什么？

错误做法是直接把问题丢给模型，让模型自由回答。

正确做法是：

- 先检索公司最新报销制度。
- 把检索到的制度片段放进上下文。
- 要求模型只能基于资料回答。
- 如果资料不足，必须回答“无法确认”。
- 输出答案时附上来源。

这就是从“让 AI 自由发挥”到“把 AI 放进工程系统”的关键变化。

Interview Questions

- AI 到底是不是在思考？为什么？
- ChatGPT 底层到底在做什么？
- AI 和传统程序最大的区别是什么？
- 为什么 AI 会产生幻觉？

- 什么是概率模型？
 - AI、机器学习、深度学习、Transformer、LLM 之间是什么关系？
 - 如果你要降低 AI 胡说的概率，工程上可以做什么？
-

Quality Checklist

- 能否用“超级实习生”的故事解释 AI 本质。
 - 能否说清 AI 不是意识，而是概率预测。
 - 能否画出 AI、ML、DL、Transformer、LLM 的层级关系。
 - 能否解释 AI 幻觉为什么发生。
 - 能否把 AI 能力放进工程约束中使用。
-

Navigation

- [Previous](#)
- [Next](#)

Chapter 03

Transformer 是什么：Attention、QKV 与生成机制

Story · Technical · Diagram · Code · Engineering · Interview

第2章：Transformer是什么（完整版教材）

本章目标

学完本章，你将彻底理解：

- Transformer 解决了什么问题
- Attention 的本质
- QKV 机制
- 为什么 RNN 被淘汰
- ChatGPT 为什么能理解上下文

1. 为什么需要Transformer？

在 Transformer 出现之前，主流序列模型是 RNN。

RNN 的思路很直观：一句话从左到右，一个词一个词读。

但 RNN 有两个致命问题：

不能并行计算

长文本容易“遗忘”

这两个问题，直接限制了模型处理长文本、长对话和复杂代码的能力。

举例理解

看这个句子：

我把苹果放在桌子上，因为它很重。

人类读到“它”的时候，知道“它”大概率指的是“苹果”。

但 RNN 要一步一步读：

- 读“我”
- 读“把”
- 读“苹果”
- 读“放在”
- 读“桌子上”
- 读“因为”
- 读“它”

问题是：

到“它”时，模型可能已经忘了前面的“苹果”。

这就叫长距离依赖问题。

2. Transformer的核心思想

一句话：

Transformer = 让模型“同时看到整个句子”

它不再像 RNN 那样只能一步一步读，而是让句子中的每个词都可以直接和其他词建立关系。

比如在这句话里：

我把苹果放在桌子上，因为它很重。

Transformer 可以直接判断：

- “它” 和 “苹果” 关系更强
- “它” 和 “桌子” 关系较弱
- “重” 和 “苹果” 关系更强

这就是 Transformer 强大的根本原因。

3. Attention机制是什么？

Attention = 注意力机制。

本质：

找到最重要的信息。

模型并不是平均看待每个词，而是会给不同词分配不同权重。

重要的信息权重大，不重要的信息权重小。

类比（非常重要）

你在读一句话时，会自动判断：

- 哪些词重要
- 哪些词不重要
- 当前词应该参考前面哪些词

例如：

苹果很重，所以它掉了下来。

在理解“它”时，“苹果”和“重”比“所以”“了”更重要。

Attention 做的就是这件事：

计算当前信息应该关注哪些历史信息。

4. QKV机制（核心）

Attention 的核心是 Q、K、V。

每个词都会生成三个向量：

角色 / 含义

Q / 我想找什么

K / 我有什么标签

V / 我提供的信息

Q、K、V 是理解 Transformer 的关键。

类比（图书馆）

你去图书馆找书。

- Q = 你想找什么书
- K = 每本书的标签
- V = 书的内容

流程是：

- 你拿着 Q 去匹配每本书的 K。
- 匹配度越高，说明这本书越相关。
- 最后根据相关程度，读取对应的 V。

这就是 Attention。

5. Attention 计算流程

简化流程如下：

$Q \times K \rightarrow$ 相似度
↓
Softmax 归一化
↓
加权 V
↓
输出结果

更直观地说：

- 用 Q 和 K 计算“相关性”。
- 用 Softmax 把相关性变成权重。

- 用权重加权 V 。
- 得到当前 token 的新表示。

6. Transformer 结构图

```
graph TD
    A[输入句子] --> B[生成 Q K V]
    B --> C[计算注意力分数]
    C --> D[Softmax]
    D --> E[加权 V]
    E --> F[输出表示]
    F --> G[后续 Transformer 层]
    G --> H[生成下一个 token]
```

7. 简化 Python 理解

下面用极简 Python 模拟 Attention 的计算过程。

```
import math

def softmax(scores):
    exp_scores = [math.exp(score) for score in scores]
    total = sum(exp_scores)
    return [score / total for score in exp_scores]

def attention(q, k, v):
    scores = [q * key for key in k]
    weights = softmax(scores)
    return sum(weight * value for weight, value in zip(weights, v))

query = 0.9
keys = [0.1, 0.8, 0.3]
values = [10, 50, 20]

print(attention(query, keys, values))
```

See also: [example.py](#)

8. 为什么 Transformer 比 RNN 强？

Transformer

- 可以并行计算
- 可以看全局
- 更擅长长距离依赖
- 更适合大规模训练
- 更适合长文本、代码和多轮对话

RNN

- 逐步计算
- 容易遗忘

- 速度慢
- 难以捕捉长距离关系

这就是为什么现代大语言模型几乎都建立在 Transformer 或其变体之上。

9. ChatGPT为什么厉害？

因为：

Transformer + 大数据 + 大参数 + 大规模训练

Transformer 提供了强大的结构，大数据提供了丰富模式，大参数提供了容量，大规模训练让模型学会语言、代码、推理和知识表达。

但请注意：

ChatGPT 能处理上下文，不代表它真正理解世界。

它的能力仍然来自模式学习和概率预测。

Engineering Use Case

假设你正在做一个“合同审查助手”。

用户上传一份很长的合同，并问：

付款条件是什么？违约责任在哪里？

如果模型不能建立长距离关系，就很难在几千字里找到关键条款。

Transformer 的 Attention

机制让模型可以在上下文中定位重要信息。但在工程系统中，你仍然不能只依赖模型直接阅读全部合同。

更可靠的做法是：

- 用 RAG 先检索相关条款。
- 把条款片段放进上下文。
- 用 Prompt 要求模型只基于条款回答。
- 输出引用位置。
- 对高风险结论加入人工审核。

10. 面试常问

Attention是什么？

计算“哪些信息更重要”的机制。

QKV是什么？

- Q：查询，表示我想找什么。

- K：键，表示我有什么标签。
- V：值，表示我提供什么信息。

Transformer解决了什么？

- 长距离依赖问题
- 不能并行的问题
- 序列模型难以捕捉全局关系的问题

为什么RNN不适合大语言模型？

因为 RNN 逐步计算，速度慢，而且长文本容易遗忘。

ChatGPT为什么能理解上下文？

因为 Transformer 的 Attention 机制能在上下文中建立 token 之间的相关性。

本章总结

- Transformer = 以 Attention 为核心的序列建模架构
- Attention = 信息加权机制
- QKV = 信息匹配系统
- Transformer 解决了 RNN 的长距离依赖和并行计算问题
- ChatGPT 的上下文能力来自 Transformer，但它仍然是概率预测系统

Quality Checklist

- 能否解释 RNN 为什么容易遗忘。
- 能否用“同时看到整个句子”解释 Transformer。
- 能否用图书馆类比解释 QKV。
- 能否说清 Attention 的计算流程。
- 能否说明 Transformer 在工程场景中的价值和边界。

Navigation

- [Previous](#)
- [Next](#)

Chapter 04

Prompt工程（如何控制AI）

Story · Technical · Diagram · Code · Engineering · Interview

第3章：Prompt工程（如何控制AI）

本章目标

学完本章，你将理解：

- 什么是 Prompt
- 为什么 Prompt 能控制 AI 输出
- System Prompt / User Prompt 的区别
- Few-shot 是什么
- CoT 是什么
- 如何让 AI 稳定输出

1. 什么是Prompt？

一句话：

Prompt = 给 AI 的指令

你对 AI 说的每一句话，本质上都是 Prompt。

比如：

帮我写一篇文章

这句话就是 Prompt。

但在工程里，Prompt 不只是“随便问一句”。它更像是一份任务说明书：告诉 AI 你是谁、要做什么、怎么做、不能做什么、最后输出成什么格式。

举例

一个很弱的 Prompt：

解释 Transformer

一个更好的 Prompt：

你是一个 AI 老师。
请用初学者能听懂的方式解释 Transformer。
要求：
- 不要使用复杂公式
- 用生活类比
- 分 3 点说明
输出格式：
1. 核心概念
2. 简单例子
3. 为什么重要

你会发现，第二个 Prompt 明显更可控。

2. 为什么Prompt重要？

因为：

AI 不会主动思考目标，它只会响应输入。

如果你的输入模糊，AI 的输出也会模糊。

如果你的输入清晰，AI 的输出就更稳定。

这就是 Prompt 工程的核心：

用清晰的语言结构，控制 AI 的输出方向。

类比

AI 像一个超级实习生：

- 你怎么说
- 他就怎么做

如果你只说：

做个分析

他可能不知道分析什么、给谁看、用什么格式、要不要给结论。

如果你说：

你是一个数据分析师，请分析这份用户反馈，找出 3 个主要问题，并按“问题-证据-建议”的格式输出。

结果就会稳定很多。

3. Prompt的结构

一个可用的 Prompt 一般包含：

- 角色 (Role)
- 任务 (Task)
- 上下文 (Context)
- 限制 (Constraint)
- 输出格式 (Format)
- 示例 (Example)

示例

你是一个资深 AI 工程师。
任务：

请解释 Transformer。

要求：

- 简单易懂
- 不要使用复杂数学公式
- 面向零基础读者

输出：

- 分点结构
- 每点不超过 100 字

这个 Prompt 比“解释 Transformer”强很多，因为它给了 AI 明确边界。

4. System vs User Prompt

类型 / 作用

System Prompt / 定义 AI 身份、规则和长期行为边界

User Prompt / 用户当前输入的任务和问题

举例

System Prompt：

你是一个 AI 老师，擅长用简单语言解释复杂技术。

User Prompt：

解释 Transformer。

System Prompt 更像“角色设定和规则”，User Prompt 更像“当前任务”。

在真实系统中，开发者通常控制 System Prompt，用户输入 User Prompt。

5. Few-shot（示例学习）

Few-shot 的意思是：

给 AI 看几个例子，让它模仿模式。

例如：

输入：苹果
输出：水果

输入：汽车
输出：交通工具

输入：Python
输出：

AI 会学习前面的模式，然后更可能输出：

编程语言

Few-shot 的价值是：与其抽象地解释规则，不如直接给 AI 看你想要的输入输出样子。

6. CoT（思维链）

CoT 是 Chain of Thought，通常翻译为“思维链”。

它的核心思想是：

让 AI 分步骤推理，而不是直接给答案。

比如：

请一步一步分析这个问题，再给出最终答案。

效果通常是：

- 更准确
- 更稳定
- 更容易发现中间错误

但在工程中要注意：不是所有场景都应该暴露完整推理过程。有些系统更适合让模型内部推理，最终只输出简洁结论和依据。

7. Prompt 工程核心技巧

Prompt 工程最重要的技巧包括：

- 明确角色
- 明确任务
- 提供上下文
- 限制输出范围
- 指定输出格式
- 提供示例
- 告诉 AI 不确定时如何回答

尤其是最后一点非常关键。

如果资料不足，你应该要求 AI 说：

无法根据现有信息确认。

而不是让它自由编造。

8. Prompt 结构模板

一个常用模板：

你是：{角色}

任务：{任务}

背景信息：
{上下文}

要求：

- {限制1}
- {限制2}
- {限制3}

输出格式：

{格式}

如果信息不足：

请回答“无法根据现有信息确认”。

这个模板可以用于总结、分类、改写、代码生成、报告生成、RAG 问答等很多场景。

Mermaid Diagram

```

graph TD
    A["System Prompt<br/>角色与规则"] --> C["LLM"]
    B["User Prompt<br/>当前任务"] --> C
    D["Context<br/>背景资料"] --> C
    E["Examples<br/>Few-shot示例"] --> C
    F["Constraints<br/>限制条件"] --> C
    C --> G["Structured Output<br/>稳定输出"]
    G --> H["Validation<br/>校验与评估"]
    H -. feedback .-> A
  
```

Python Code

下面用 Python 构造一个 Prompt 模板。

```

from string import Template

prompt_template = Template("""
你是：$role

任务：$task

背景信息：
$context

要求：
- $constraint_1
- $constraint_2

输出格式：
$output_format

如果信息不足，请回答：无法根据现有信息确认。
""")

prompt = prompt_template.substitute(
    role="资深 AI 工程师",
    task="解释 Transformer",
    context="读者是零基础开发者",
    constraint_1="不要使用复杂公式",
    constraint_2="使用生活类比",
    output_format="分点输出：核心概念、简单例子、为什么重要",
)

print(prompt)
  
```

See also: [example.py](#)

Engineering Use Case

假设你正在做一个客服质检系统。

用户输入是一段客服对话，你希望 AI 输出：

- 问题分类
- 严重程度
- 证据片段
- 建议动作

错误做法是：

分析这段客服对话。

正确做法是：

你是客服质检专家。
请分析以下客服对话。

要求：

- 只能基于对话内容判断
- 如果证据不足，返回 unknown
- 输出 JSON

JSON 字段：

- category
- severity
- evidence
- next_action

这就是 Prompt 从“聊天”变成“工程接口”的关键。

9. 面试常问

什么是 Prompt？

Prompt 是控制 AI 输出的输入指令。

Few-shot 是什么？

Few-shot 是通过示例教 AI 模仿输入输出模式。

为什么 Prompt 重要？

因为 AI 完全依赖输入。输入越清晰，输出越稳定。

System Prompt 和 User Prompt 有什么区别？

System Prompt 定义角色和规则，User Prompt 提供当前任务。

如何让 AI 稳定输出？

明确角色、任务、限制、格式、示例，并加入输出校验。

本章总结

- Prompt = 控制 AI 的方式
 - 结构越清晰，输出越稳定
 - System Prompt 定义身份和规则
 - User Prompt 定义当前任务
 - Few-shot 用例子教 AI
 - CoT 让 AI 分步骤分析
 - Prompt 本质上是指挥 AI 的语言
-

Quality Checklist

- 能否解释 Prompt 为什么能控制 AI 输出。
 - 能否区分 System Prompt 和 User Prompt。
 - 能否写出包含角色、任务、限制和格式的 Prompt。
 - 能否解释 Few-shot 和 CoT。
 - 能否把 Prompt 设计成工程接口，而不是随便聊天。
-

Navigation

- [Previous](#)
- [Next](#)

Chapter 05

RAG（检索增强生成）

Story · Technical · Diagram · Code · Engineering · Interview

第4章：RAG（检索增强生成）

本章目标

学完本章你将理解：

- 什么是RAG
- 为什么LLM需要RAG
- 向量数据库在做什么
- Embedding是什么
- RAG如何减少幻觉
- 企业如何用RAG

1. 为什么需要RAG？

LLM有一个问题：

它不知道“最新知识”

例如：

- 最新公司制度
- 最新股票价格
- 私有文档内容

举例

你问：

公司年假是多少天？

LLM可能：

编一个答案（幻觉）

解决方案：

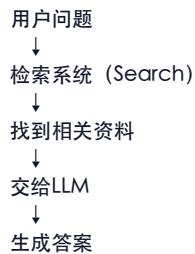
RAG（Retrieval Augmented Generation）

2. RAG是什么？

一句话：

RAG = 先查资料，再回答

流程



3. RAG核心结构

RAG由三部分组成：

- Embedding（向量化）
- Vector Database（向量数据库）
- LLM（生成）

4. Embedding是什么？

Embedding = 把文字变成数字

类比

一句话：

“苹果很好吃”

变成：

[0.12, -0.98, 0.33, ...]

为什么要这样？

因为：

计算机只能处理数字

5. 向量数据库在做什么？

它做一件事：

找“最相似的内容”

类比

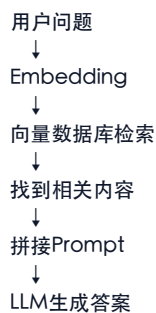
你问：

什么是AI？

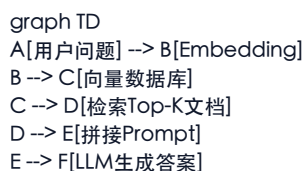
系统会找：

- AI介绍文章
- 机器学习文档
- Transformer解释

6. RAG完整流程



7. RAG架构图



8. 为什么RAG能减少幻觉？

因为：

- LLM不再“猜”
- 而是“有资料参考”

没RAG：

LLM = 闭卷考试

有RAG：

LLM = 开卷考试

9. 企业为什么用RAG？

因为：

- 可以接私有数据
- 不用重新训练模型
- 成本低
- 更新快

10. 极简代码理解

```
def rag(query, docs):  
    related_docs = search(docs, query)  
    context = "\n".join(related_docs)  
    return llm(query + context)
```

11. 面试常问

什么是RAG？

检索增强生成，让LLM先查资料再回答

RAG解决什么问题？

- 幻觉问题
- 知识更新问题

Embedding是什么？

将文本转换为向量表示

本章总结

- RAG = 检索 + LLM生成
- Embedding = 文本向量化

- Vector DB = 相似度搜索

Engineering Use Case

企业知识库问答是 RAG 最典型的落地场景。员工问“公司年假是多少天”，系统不应该让 LLM 自由猜，而应该先检索公司制度文档，再把相关片段交给 LLM，最后输出答案和引用来源。

Mermaid Diagram

```
graph TD
  A[用户问题] --> B[Embedding]
  B --> C[向量数据库]
  C --> D[检索Top-K文档]
  D --> E[拼接Prompt]
  E --> F[LLM生成答案]
  F --> G[引用来源与答案]
```

Python Code

```
def search(docs, query):
    query_terms = set(query.lower().split())
    scored = []
    for doc in docs:
        terms = set(doc.lower().split())
        score = len(query_terms & terms)
        scored.append((score, doc))
    return [doc for score, doc in sorted(scored, reverse=True) if score > 0]

def llm(prompt):
    return "基于资料回答：" + prompt[:120]

def rag(query, docs):
    related_docs = search(docs, query)
    context = "\n".join(related_docs)
    prompt = f"问题：{query}\n资料：{context}"
    return llm(prompt)

docs = [
    "公司 年假 政策 员工 每年 10 天",
    "报销 需要 发票 和 审批",
    "AI Agent 使用 RAG 查询 私有知识",
]
print(rag("公司 年假 政策", docs))
```

See also: [example.py](#)

Quality Checklist

- 能否用一句话解释本章核心概念。
- 能否画出本章系统流程。
- 能否运行 Python 示例理解核心机制。
- 能否说明企业工程场景如何落地。

- 能否回答本章高频面试题。
-

Navigation

- [Previous](#)
- [Next](#)

Chapter 06

Agent（AI如何真正干活）

Story · Technical · Diagram · Code · Engineering · Interview

第5章：Agent（AI如何真正“干活”）

本章目标

学完本章你将理解：

- 什么是Agent
- 为什么LLM本身不能执行任务
- Tool / Function Calling是什么
- ReAct是什么
- AI如何“思考+行动”
- 多Agent系统是什么

1. 什么是Agent？

一句话：

Agent = 能思考 + 能执行任务的AI系统

对比理解

类型 / 能力

LLM / 只能回答问题

Agent / 能执行任务

举例

LLM：

你说：

帮我订机票

它回答：

可以去携程订票

Agent：

它会：

- 打开网页

- 搜索航班
- 选择时间
- 填写信息
- 完成订票

2. 为什么需要Agent？

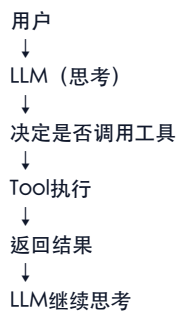
因为：

LLM只有“脑子”，没有“手”

Agent =

大脑（LLM）+ 手（工具）

3. Agent结构



4. Tool / Function Calling

Tool Calling = 让AI使用外部工具

举例

用户：

查一下北京天气

LLM输出：

```
{
  "tool": "weather",
  "location": "北京"
}
```

系统执行：

调用天气API

返回结果：

北京今天 18° C

5. ReAct（核心方法）

ReAct =

Reasoning（思考） + Acting（行动）

流程

思考 → 行动 → 观察 → 再思考 → 再行动

类比

像一个人：

- 想一步
- 做一步
- 看结果
- 再调整

6. Agent工作循环

Think → Act → Observe → Repeat

7. Agent架构图

```
graph TD
  A[用户输入] --> B[LLM思考]
  B --> C[选择工具]
  C --> D[执行Tool]
  D --> E[返回结果]
  E --> B
```

8. 多Agent系统

多个Agent协作：

举例

Agent / 作用

Planner / 规划任务

Writer / 写内容

Reviewer / 检查

Tool Agent / 执行操作

类比公司

- CEO（规划）
- 员工（执行）
- 审核（质量控制）

9. Agent vs LLM

对比 / LLM / Agent

能否行动 //

是否调用工具 //

是否有循环 //

10. 简化代码理解

```
def agent(user_input):  
    thought = llm(user_input)  
  
    if need_tool(thought):  
        result = tool(thought)  
        return llm(result)  
  
    return thought
```

11. Agent的本质

一句话：

Agent = 会使用工具的LLM

12. 面试常问

什么是Agent？

能使用工具并完成任务的AI系统

Agent和LLM区别？

- LLM：只能回答
- Agent：能执行任务

ReAct是什么？

思考 + 行动的循环机制

本章总结

- Agent = LLM + 工具 + 循环
- ReAct = 思考+行动
- Tool Calling = AI使用外部能力

Engineering Use Case

Agent 适合处理需要多步骤行动的任务，比如“分析销售数据并生成报告”。LLM 负责规划和判断，工具负责查询数据、生成文件、发送消息，系统负责记录状态和控制权限。

Mermaid Diagram

```
graph TD
  A[用户输入] --> B[LLM思考]
  B --> C[选择工具]
  C --> D[执行Tool]
  D --> E[返回结果]
  E --> B
  B --> F[最终答案]
```

Python Code

```
def llm(user_input):
    if "天气" in user_input:
        return {"need_tool": True, "tool": "weather", "args": {"city": "北京"}}
    return {"need_tool": False, "answer": "直接回答用户问题"}

def weather_tool(city):
    return f"{city} 今天 18° C"

def agent(user_input):
    thought = llm(user_input)
    if thought["need_tool"]:
        result = weather_tool(thought["args"]["city"])
        return f"工具结果: {result}。最终回答: 北京今天 18° C。"
```



```
return thought["answer"]
```

```
print(agent("查一下北京天气"))
```

See also: [example.py](#)

Quality Checklist

- 能否用一句话解释本章核心概念。
 - 能否画出本章系统流程。
 - 能否运行 Python 示例理解核心机制。
 - 能否说明企业工程场景如何落地。
 - 能否回答本章高频面试题。
-

Navigation

- [Previous](#)
- [Next](#)

Chapter 07

Tools & Function Calling (让AI真正动手)

Story · Technical · Diagram · Code · Engineering · Interview

第6章：Tools & Function Calling（让AI真正“动手”）

本章目标

学完本章你将理解：

- 什么是Function Calling
- 为什么AI需要工具
- JSON Schema在做什么
- Tool调用完整流程
- Agent如何连接外部世界
- 企业如何使用工具系统

1. 为什么需要Tools？

LLM有一个核心问题：

它只能“说”，不能“做”

举例

你说：

查一下北京天气

LLM：

北京天气很好（可能是编的）

问题本质：

LLM没有：

- 互联网
- 数据库
- API
- 系统权限

2. Tools是什么？

一句话：

Tools = 给AI “外部能力”

类比

AI = 大脑 Tools = 手 + 工具

3. Function Calling是什么？

Function Calling =

AI决定调用哪个函数 + 传什么参数

示例

用户：

查北京天气

LLM输出：

```
{
  "function": "get_weather",
  "arguments": {
    "city": "北京"
  }
}
```

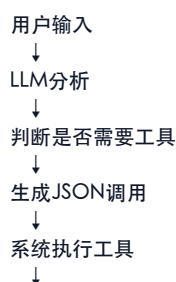
系统执行：

调用 weather API

返回：

北京 18° C

4. Tool调用流程



返回结果给LLM
↓
LLM生成最终回答

5. Tool调用架构图

```
graph TD
  A[用户输入] --> B[LLM]
  B --> C{是否需要工具}
  C --> |是| D[Function Call JSON]
  D --> E[执行API/工具]
  E --> F[返回结果]
  F --> B
  B --> G[最终回答]
```

6. JSON Schema是什么？

JSON Schema =

定义函数输入格式的规则

示例

```
{
  "name": "get_weather",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {
        "type": "string"
      }
    }
  }
}
```

7. 为什么需要Schema？

因为：

- 防止AI乱输出
- 保证参数正确
- 方便系统执行

8. Tool vs Prompt区别

类型 / 作用

Prompt / 控制AI说什么

Tool / 让AI做什么

9. 多Tool系统

一个Agent可以用多个工具：

示例

- 天气API
- 搜索引擎
- 数据库
- 计算器
- 邮件系统

类比

AI = 操作系统 Tools = 软件应用

10. 企业级Tool系统

企业Agent通常包含：

- CRM系统
- ERP系统
- 数据库
- API网关
- 搜索系统

11. 简化代码理解

```
def function_call(user_input):  
    if "天气" in user_input:  
        return call_weather_api()  
    else:  
        return llm(user_input)
```

12. Tool Calling的本质

一句话：

让AI具备“执行能力”

13. 面试常问

什么是Function Calling？

AI通过结构化JSON调用外部函数

Tool有什么作用？

扩展AI能力边界

为什么需要Schema？

保证参数结构正确

本章总结

- Tool = AI外部能力
- Function Calling = AI调用函数
- Schema = 结构约束
- Agent = LLM + Tools

Engineering Use Case

企业 Agent 通常会把 CRM、ERP、数据库、搜索引擎、邮件系统都封装成工具。Function Calling 的关键不是让模型直接执行，而是让模型输出结构化调用意图，由系统校验参数后执行。

Mermaid Diagram

```
graph TD
  A[用户输入] --> B[LLM]
  B --> C{是否需要工具}
  C --> |是| D[Function Call JSON]
  D --> E[执行API/工具]
  E --> F[返回结果]
  F --> B
  C --> |否| G[直接回答]
  B --> H[最终回答]
```

Python Code

```
def call_weather_api(city):
```

```
return {"city": city, "temperature": "18° C", "condition": "晴"}

def llm(user_input):
    return f"普通回答：{user_input}"

def function_call(user_input):
    if "天气" in user_input:
        tool_call = {"function": "get_weather", "arguments": {"city": "北京"}}
        result = call_weather_api(tool_call["arguments"]["city"])
        return f"{result['city']} 今天 {result['temperature']}，天气{result['condition']}"
    return llm(user_input)

print(function_call("查一下北京天气"))
```

See also: [example.py](#)

Quality Checklist

- 能否用一句话解释本章核心概念。
- 能否画出本章系统流程。
- 能否运行 Python 示例理解核心机制。
- 能否说明企业工程场景如何落地。
- 能否回答本章高频面试题。

Navigation

- [Previous](#)
- [Next](#)

Chapter 08

LangGraph（AI Agent编排系统）

Story · Technical · Diagram · Code · Engineering · Interview

第7章：LangGraph（AI Agent编排系统）

本章目标

学完本章你将理解：

- 为什么需要LangGraph
- 什么是Graph结构
- Node / Edge / State是什么
- 如何编排多个Agent
- 为什么Agent需要“流程控制”
- 企业级Agent如何设计

1. 为什么需要LangGraph？

在前面你已经学过：

- LLM（只会回答）
- Agent（会调用工具）
- Tool Calling（能执行动作）

但是问题来了：

Agent是“无序执行”的

举例

一个复杂任务：

写一份电商分析报告

需要：

- 搜索数据
- 分析数据
- 写报告
- 检查质量

如果没有结构：

Agent会乱执行 顺序不稳定 结果不可控

2. LangGraph是什么？

一句话：

LangGraph = 用“图结构”控制Agent流程

核心思想

不是线性执行，而是：

Graph（图结构）执行

3. Graph结构

Node = 执行步骤
Edge = 流程连接
State = 共享数据

4. LangGraph结构图

```
graph TD
  A[Start] --> B[Planner Node]
  B --> C[Tool Node]
  C --> D[LLM Node]
  D --> E[Reviewer Node]
  E --> F[End]
```

5. 三个核心概念

1 Node（节点）

一个执行单元：

- LLM
- Tool
- Function
- Agent

2 Edge（边）

控制流程：

- A → B

- B → C

3 State（状态）

全局数据：

- 用户输入
- 中间结果
- 工具返回

6. 为什么需要Graph？

因为：

LLM + Agent 的问题是：

- 无法控制流程
- 容易乱跳步骤
- 不稳定

Graph解决：

可控流程
可重复执行
可调试

7. Agent vs LangGraph

类型 / 特点

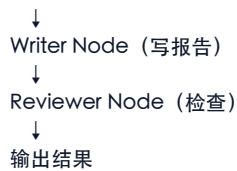
Agent / 自由执行

LangGraph / 可控流程

8. 一个真实流程

电商分析Agent：





9. 简化代码理解

```
state = {}

def node1(state):
    state["plan"] = "search data"
    return state

def node2(state):
    state["data"] = "result"
    return state
```

10. LangGraph的本质

一句话：

LangGraph = 可控的Agent系统执行框架

11. 为什么企业喜欢LangGraph？

因为：

- 可控
- 可调试
- 可扩展
- 可监控

12. 面试常问

LangGraph是什么？

用图结构控制AI Agent流程的框架

Node和Edge是什么？

- Node = 执行步骤
- Edge = 流程连接

为什么不用普通Agent？

因为不稳定、不可控

本章总结

- LangGraph = 图结构Agent
- Node = 执行单元
- Edge = 流程
- State = 数据

Engineering Use Case

复杂 Agent 任务不能只靠自由循环。电商分析报告可以拆成

Planner、Search、Analysis、Writer、Reviewer 五个节点，每个节点读写同一个 State，让流程可控、可调试、可恢复。

Mermaid Diagram

```
graph TD
  A[Start] --> B[Planner Node]
  B --> C[Tool Node]
  C --> D[LLM Node]
  D --> E[Reviewer Node]
  E --> F[End]
```

Python Code

```
state = {}

def planner_node(state):
    state["plan"] = "search data -> analyze -> write report"
    return state

def tool_node(state):
    state["data"] = "sales increased, conversion dropped"
    return state

def writer_node(state):
    state["report"] = f"Plan: {state['plan']} | Data: {state['data']}"
    return state

for node in (planner_node, tool_node, writer_node):
    state = node(state)

print(state["report"])
```

See also: [example.py](#)

Quality Checklist

- 能否用一句话解释本章核心概念。
 - 能否画出本章系统流程。
 - 能否运行 Python 示例理解核心机制。
 - 能否说明企业工程场景如何落地。
 - 能否回答本章高频面试题。
-

Navigation

- [Previous](#)
- [Next](#)

Chapter 09

MCP (Model Context Protocol)

Story · Technical · Diagram · Code · Engineering · Interview

第8章：MCP（Model Context Protocol）

本章目标

学完本章你将理解：

- MCP是什么
- 为什么需要统一协议
- Agent如何连接工具生态
- MCP和Function Calling区别
- 企业为什么要用MCP
- AI工具化的未来架构

1. MCP是什么？

一句话：

MCP = AI连接工具的“统一协议”

类比理解

你可以把MCP想象成：

USB-C接口

在MCP之前：

每个工具都要单独对接：

- OpenAI tools
- Claude tools
- LangChain tools
- 自定义API

非常混乱

MCP之后：

统一接口：

一个协议 → 所有工具通用

2. 为什么需要MCP？

因为现在AI工具生态很乱：

- 每个模型接口不一样
- 每个工具调用方式不同
- 难以扩展
- 难以维护

没有MCP的问题

- 工具无法复用
- Agent无法跨平台
- 开发成本极高

MCP解决：

标准化AI与工具通信

3. MCP核心结构

MCP由三部分组成：

Client（AI）
Server（工具提供方）
Protocol（通信协议）

4. MCP架构图

```
graph TD
  A[AI Model] --> B[MCP Client]
  B --> C[MCP Server]
  C --> D[Tools/API/DB]
  D --> C
  C --> B
  B --> A
```

5. MCP vs Function Calling

对比 / Function Calling / MCP

目标 / 单模型工具调用 / 标准化生态

范围 / 单应用 / 多系统

扩展性 / 弱 / 强

标准 / 无统一 / 有协议

6. MCP解决什么问题？

以前：

每个Agent都要写：

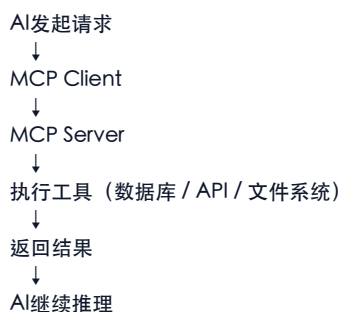
- tool接口
- API格式
- 参数结构

MCP之后：

只需要：

插拔式工具系统

7. MCP工作流程



8. MCP的本质

一句话：

MCP = AI时代的“工具操作系统”

9. MCP的应用场景

1. AI编程

- 读代码
- 改代码
- 执行测试

2. 数据分析

- 查询数据库
- 生成报表
- 可视化

3. 企业系统

- CRM
- ERP
- BI系统

10. MCP vs Agent

类型 / 作用

Agent / 决策

MCP / 工具连接

本质关系：

Agent = 大脑

MCP = 神经接口

11. 为什么MCP很重要？

因为未来AI不是单模型，而是：

多模型 + 多工具 + 多系统

12. MCP未来趋势

未来AI系统会变成：

- 插件化
- 工具化

- 服务化

13. 面试常问

MCP是什么？

AI与工具之间的标准通信协议

MCP解决什么问题？

- 工具碎片化
- 系统不统一
- 难扩展

MCP和Function Calling区别？

- Function Calling：单点能力
- MCP：生态协议

本章总结

- MCP = AI工具统一协议
- 解决工具碎片化问题
- 让AI系统可扩展

Engineering Use Case

MCP 适合把企业工具做成统一协议层。不同 AI 客户端不需要分别对接 CRM、数据库、文件系统，而是通过 MCP Server 发现工具、读取资源、执行受控操作。

Mermaid Diagram

```
graph TD
  A[AI Model] --> B[MCP Client]
  B --> C[MCP Server]
  C --> D[Tools/API/DB]
  D --> C
  C --> B
  B --> A
```

Python Code

```
mcp_server = {
    "name": "company-tools",
    "tools": [
        {"name": "search_docs", "description": "搜索企业文档"},
        {"name": "query_crm", "description": "查询客户系统"},
    ],
    "resources": ["docs://policies", "crm://customers"],
}

for tool in mcp_server["tools"]:
    print(f"MCP Tool: {tool['name']} - {tool['description']}")
```

See also: [example.py](#)

Quality Checklist

- 能否用一句话解释本章核心概念。
- 能否画出本章系统流程。
- 能否运行 Python 示例理解核心机制。
- 能否说明企业工程场景如何落地。
- 能否回答本章高频面试题。

Navigation

- [Previous](#)
- [Next](#)

Chapter 10

AI Agent 实战项目（电商 / CRM / 教师）

Story · Technical · Diagram · Code · Engineering · Interview

第9章：AI Agent实战项目（电商 / CRM / 教师）

本章目标

学完本章你将能够：

- 理解真实AI Agent如何落地
- 设计一个完整AI系统
- 构建电商/CRM/教育Agent
- 理解企业级AI架构
- 将前面所有知识串起来

1. 为什么要做实战？

前面你学的是：

- Transformer（理解模型）
- RAG（知识增强）
- Agent（行动能力）
- Tool（工具调用）
- MCP（工具协议）

但这些都是“单点能力”

这一章做什么？

把所有能力组合成真实系统

2. 电商AI Agent（完整案例）

功能

电商AI Agent可以：

- 分析店铺数据
- 生成运营报告
- 推荐优化策略

- 自动写PPT
- 自动生成总结

架构

```
graph TD
  A[店铺数据] --> B[RAG检索]
  B --> C[Agent分析]
  C --> D[生成策略]
  D --> E[生成报告]
  E --> F[PPT输出]
```

流程

```
graph TD
  A[用户上传店铺数据] --> B[数据解析 CSV / Excel]
  B --> C[RAG检索行业经验]
  C --> D[Agent分析问题]
  D --> E[生成优化方案]
  E --> F[输出报告]
```

3. CRM Agent（客户管理系统）

功能

- 客户分析
- 销售建议
- 自动写邮件
- 跟进提醒

架构

```
graph TD
  A[客户数据] --> B[CRM Agent]
  B --> C[客户分层]
  C --> D[销售策略]
  D --> E[自动邮件生成]
```

应用

- B端销售系统
- SaaS客户管理

- 自动营销系统

4. 教师AI Agent（教育场景）

功能

- 批改作业
- 生成题目
- 学生分析
- 学习建议

架构

```
graph TD
  A[作业图片] --> B[OCR识别]
  B --> C[AI批改]
  C --> D[错误分析]
  D --> E[生成讲解]
```

5. 三大系统统一架构

本质一样

数据 → RAG → Agent → Tool → 输出

总架构

```
graph TD
  A[数据输入] --> B[RAG系统]
  B --> C[Agent推理]
  C --> D[Tools执行]
  D --> E[结果输出]
```

6. 企业级AI系统特点

1. 多数据源

- Excel
- API
- 数据库

- 文档

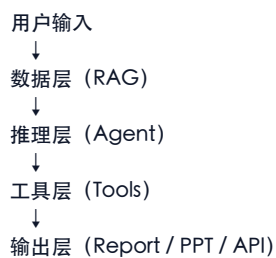
2. 多Agent协作

- Planner
- Analyst
- Writer
- Reviewer

3. 工具系统

- MCP
- Function Calling
- API系统

7. 一个完整AI系统



8. 为什么这个很重要？

因为：

所有AI产品 = 这些模块组合

9. 面试必问

如何设计一个AI Agent系统？

答：

- RAG + Agent + Tools + Workflow

电商AI系统怎么做？

- 数据分析
- RAG行业知识
- Agent生成策略
- Tools生成报告

CRM AI怎么做？

- 客户数据分析
- 分层
- 自动营销

本章总结

- AI系统 = 多模块组合
- RAG + Agent + Tools = 核心架构
- 企业AI = 流程化系统

Engineering Use Case

一个完整 AI Agent 项目不是单点能力，而是数据层、RAG 层、Agent 推理层、Tools 执行层和输出层的组合。电商、CRM、教育场景虽然业务不同，但底层架构高度相似。

Mermaid Diagram

```
graph TD
  A[数据输入] --> B[RAG系统]
  B --> C[Agent推理]
  C --> D[Tools执行]
  D --> E[结果输出]
  E --> F[报告/PPT/API]
```

Python Code

```
def ecommerce_agent(data):
    rag_context = "行业经验：提升转化率，优化商品标题和主图"
    analysis = f"店铺数据：{data}; 参考：{rag_context}"
    strategy = "建议：优化爆款商品详情页，并针对低转化流量做人群分层。"
    return {"analysis": analysis, "strategy": strategy, "output": "运营报告"}

result = ecommerce_agent("访客上涨，转化率下降")
print(result["strategy"])
```

See also: [example.py](#)

Quality Checklist

- 能否用一句话解释本章核心概念。
 - 能否画出本章系统流程。
 - 能否运行 Python 示例理解核心机制。
 - 能否说明企业工程场景如何落地。
 - 能否回答本章高频面试题。
-

Navigation

- [Previous](#)
- [Next](#)

Chapter 11

AI面试终极宝典 (LLM / RAG / Agent / MCP)

Story · Technical · Diagram · Code · Engineering · Interview

第10章：AI面试终极宝典（LLM / RAG / Agent / MCP）

本章目标

学完本章你将能够：

- 应对AI工程师面试
- 理解高频技术问题
- 讲清Transformer / RAG / Agent
- 做系统设计题
- 用项目经验回答问题

1. 面试核心逻辑

面试官真正想考的是：

你是否能设计一个AI系统

而不是：

- 公式
- 死记知识

2. Transformer面试题

Q1：Transformer是什么？

答：

基于Attention机制的序列建模模型

Q2：为什么比RNN强？

- 可并行
- 长距离依赖
- 更稳定

Q3：Attention是什么？

计算不同token之间重要性的机制

Q4：QKV是什么？

- Q：查询
- K：键
- V：值

3. RAG面试题

什么是RAG？

检索增强生成（Retrieval Augmented Generation）

为什么需要RAG？

- 减少幻觉
- 引入外部知识
- 支持私有数据

RAG流程？

用户 → Embedding → 向量数据库 → LLM → 输出

Embedding是什么？

把文本转换为向量表示

4. Agent面试题

什么是Agent？

能调用工具并执行任务的LLM系统

Agent vs LLM区别？

LLM / Agent

只能回答 / 能执行

无工具 / 有工具

ReAct是什么？

思考 + 行动的循环机制

5. MCP面试题

MCP是什么？

AI工具调用的统一协议

为什么需要MCP？

- 统一工具接口
- 解决碎片化问题
- 提高可扩展性

MCP vs Function Calling？

Function Calling / MCP

单模型能力 / 工具生态协议

6. 系统设计题（最重要）

设计一个AI客服系统

答：

RAG + Agent + Tools + CRM

设计一个电商AI系统

结构：

- 数据层（订单/商品）
- RAG（行业知识）
- Agent（策略生成）

- Tools (API/PPT生成)

设计一个AI写书系统

结构：

- Planner Agent
- Writer Agent
- Reviewer Agent
- Markdown输出
- GitHub部署

7. 项目面试话术

你做过什么AI项目？

答：

我做了一个基于RAG + Agent的电商分析系统

包含：

- 数据分析
- 自动报告
- 策略生成

你的亮点？

- 多Agent系统
- 工具调用
- RAG增强
- 实战项目

8. 高频总结（必须背）

- Transformer = Attention
- RAG = 检索 + 生成
- Agent = LLM + Tools

- MCP = 工具协议

本章总结

AI工程本质 = 系统设计能力 + 模块组合能力

Engineering Use Case

AI 面试的核心不是背定义，而是把 Transformer、RAG、Agent、MCP 和项目经验组织成系统设计答案。最好的回答能讲清模块组合、工程取舍和真实落地风险。

Mermaid Diagram

```
graph TD
  A[AI面试] --> B[Transformer]
  A --> C[RAG]
  A --> D[Agent]
  A --> E[MCP]
  A --> F[系统设计]
  F --> G[项目经验]
```

Python Code

```
answer_framework = [
    "先澄清业务目标和用户场景",
    "再说明系统模块：RAG + Agent + Tools + Workflow",
    "然后解释数据流、工具流和评估指标",
    "最后说明风险控制、成本和迭代方案",
]

for step in answer_framework:
    print("-", step)
```

See also: [example.py](#)

Quality Checklist

- 能否用一句话解释本章核心概念。
- 能否画出本章系统流程。
- 能否运行 Python 示例理解核心机制。
- 能否说明企业工程场景如何落地。
- 能否回答本章高频面试题。

Navigation

- [Previous](#)

- Next